```
!pip install pandas numpy scikit-learn seaborn matplotlib shap xgboost lightgbm optuna
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.51.0)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: lightgbm in /usr/local/lib/python3.12/dist-packages (4.6.0)
Collecting optuna
  Downloading optuna-4.8.0-py3-none-any.whl.metadata (17 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: alembic>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from optuna) (1.18.4)
Collecting colorlog (from optuna)
  Downloading colorlog-6.10.1-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: sqlalchemy>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from optuna) (2.0.48)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages (from optuna) (6.0.3)
Requirement already satisfied: Mako in /usr/local/lib/python3.12/dist-packages (from alembic>=1.5.0->optuna) (1.3.10)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy>=1.4.2->optuna) (3.3.2)
Requirement already satisfied: MarkupSafe>=0.9.2 in /usr/local/lib/python3.12/dist-packages (from Mako->alembic>=1.5.0->optuna
Downloading optuna-4.8.0-py3-none-any.whl (419 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 419.5/419.5 kB 7.0 MB/s eta 0:00:00
Downloading colorlog-6.10.1-py3-none-any.whl (11 kB)
Installing collected packages: colorlog, optuna
Successfully installed colorlog-6.10.1 optuna-4.8.0
```

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import f1_score

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from xgboost import XGBClassifier

import shap
import optuna
import pickle

import warnings
warnings.filterwarnings("ignore")
```

Double-click (or enter) to edit

```
df = pd.read_csv('/content/tv-shows.csv')
df.head()
```

	id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description	pl
0	1	Movie	ChuChuTV Surprise Eggs Learning Videos (English)	NaN	NaN	NaN	October 18, 2019	2019	TV-Y	61 min	Children & Family Movies	From colors and letters to animals of all kind...	
1	2	Movie	The Journey Is the Destination	Bronwen Hughes	Ben Schnetzer, Kelly Macdonald, Sam Hazeldine,...	United States	November 7, 2017	2016	R	123 min	Dramas	Spirited 22-year-old activist and photojournal...	
2	3	TV Show	Champions	NaN	Anders Holm, Fortune Feimster, Andy Favreau, J...	United States	June 19, 2021	2018	TV-14	1 Season	TV Comedies	Years after getting his girlfriend pregnant, w...	
3	4	TV Show	The Returned	NaN	Anne Cosigny, Frédéric Pierrot, Clotilde Hesme...	France	February 17, 2019	2015	TV-MA	1 Season	International TV Shows, TV Dramas, TV Horror	On returning home and finding they're believed...	
4	5	Movie	Super Bheem Bana Vajraveer	Sumit Das	Sonal Kaushal, Rupa Bhimani, Julie Tejwani, Sa...	India	June 18, 2019	2018	TV-Y7	63 min	Children & Family Movies	Hoping to find a magical root, a monster has c...	

Next steps: Generate code with df New interactive sheet

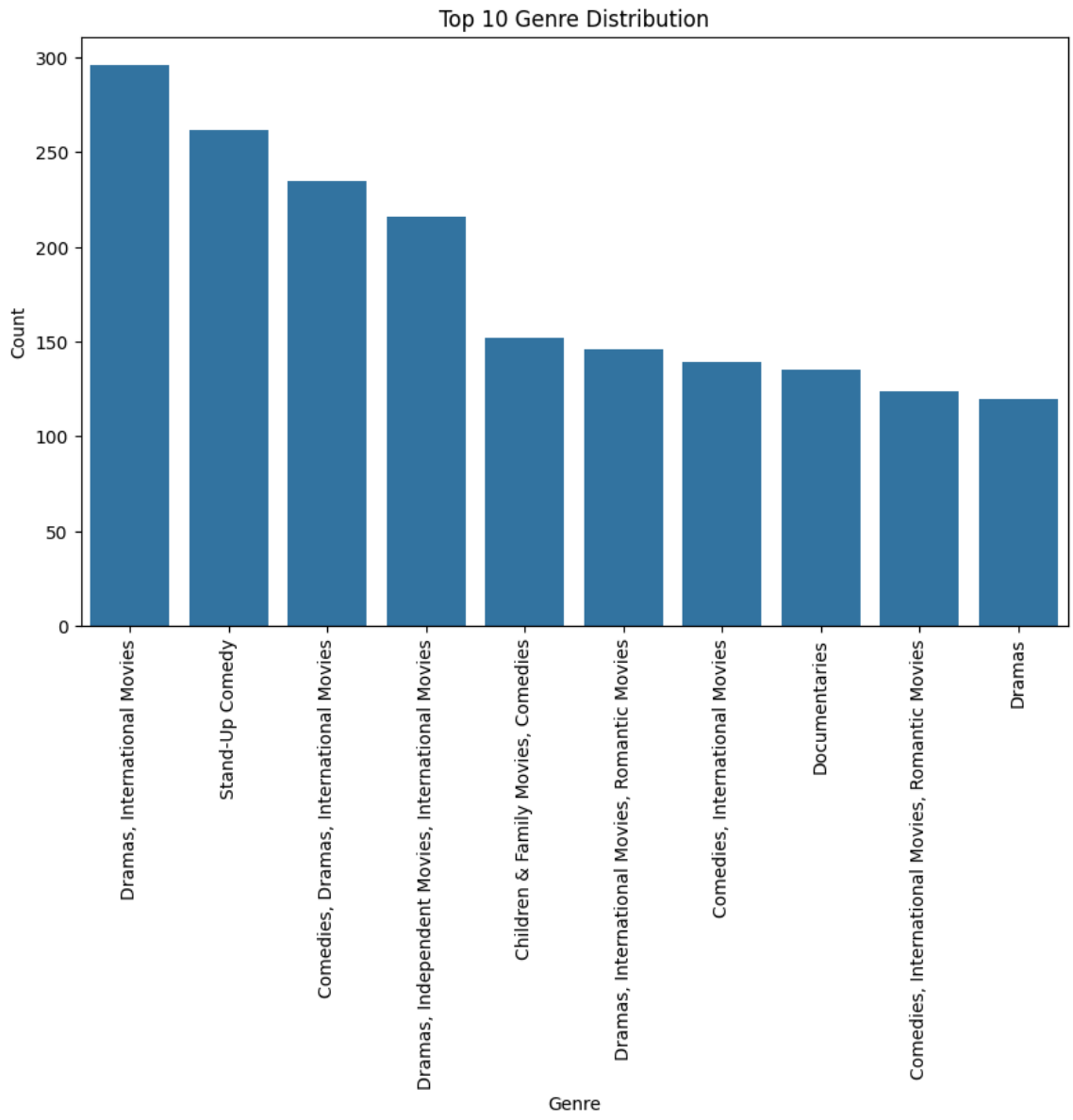
```
print("Dataset Shape:", df.shape)
df.info()
df.describe()
```

```
Dataset Shape: (9338, 13)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9338 entries, 0 to 9337
Data columns (total 13 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   id            9338 non-null   int64
 1   type          9338 non-null   object
 2   title         9338 non-null   object
 3   director      6483 non-null   object
 4   cast          8400 non-null   object
 5   country       8376 non-null   object
 6   date_added    9325 non-null   object
 7   release_year  9338 non-null   int64
 8   rating        9331 non-null   object
 9   duration      9335 non-null   object
 10  listed_in     9338 non-null   object
 11  description   9338 non-null   object
 12  platform      9338 non-null   object
dtypes: int64(2), object(11)
memory usage: 948.5+ KB
```

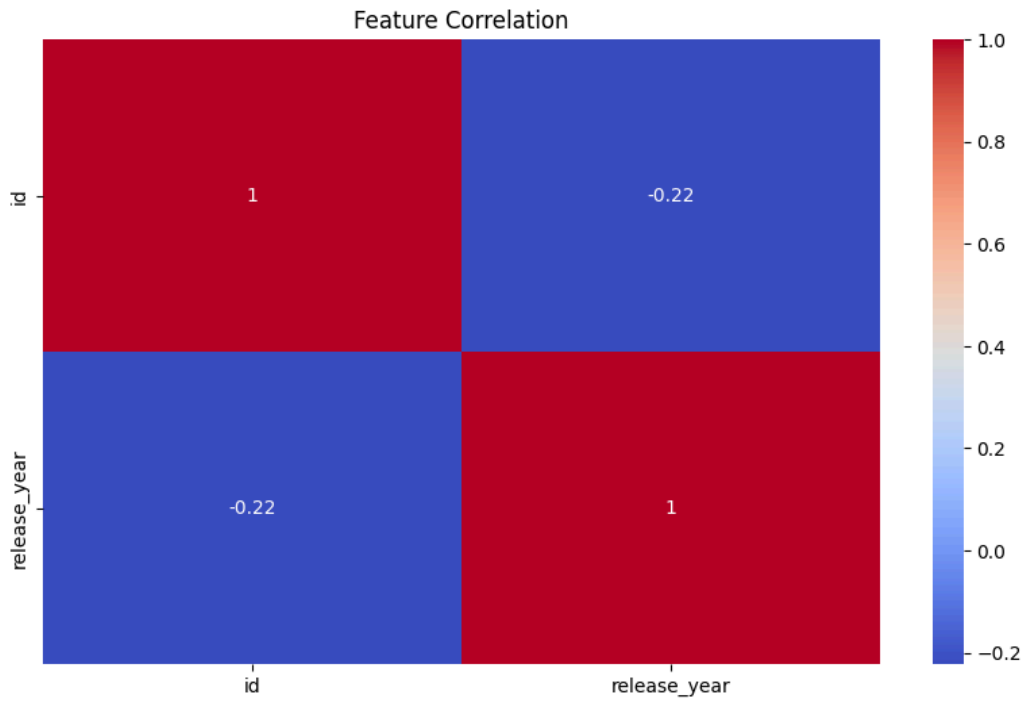
	id	release_year
count	9338.000000	9338.000000
mean	4669.500000	2012.462305
std	2695.792741	12.512379
min	1.000000	1928.000000
25%	2335.250000	2012.000000
50%	4669.500000	2017.000000
75%	7003.750000	2019.000000
max	9338.000000	2021.000000

```
df.isnull().sum()
df = df.dropna()
```

```
plt.figure(figsize=(10,6))
sns.countplot(x=df['listed_in'], order = df['listed_in'].value_counts().index[:10])
plt.xticks(rotation=90)
plt.title("Top 10 Genre Distribution")
plt.xlabel("Genre")
plt.ylabel("Count")
plt.show()
```




```
plt.figure(figsize=(10,6))
sns.heatmap(df.select_dtypes(include=np.number).corr(),annot=True,cmap='coolwarm')
plt.title("Feature Correlation")
plt.show()
```




```
# Work on a copy to avoid modifying the original df too early
df_working = df.copy()
df_working = df_working.drop(columns=['id']) # Drop only 'id' for now

label_encoders = {}
for col in df_working.select_dtypes(include='object').columns:
    le = LabelEncoder()
    df_working[col] = le.fit_transform(df_working[col])
    label_encoders[col] = le

# Now df_working has all object columns (including title and listed_in) encoded.

y_initial_encoded = df_working['listed_in']

# Identify and remove classes with less than 2 members for stratification
class_counts = y_initial_encoded.value_counts()
classes_to_keep = class_counts[class_counts >= 2].index # Corrected line

# Filter df_working based on classes_to_keep
df_filtered_common = df_working[y_initial_encoded.isin(classes_to_keep)].copy()

# Re-encode y_filtered to ensure contiguous 0-indexed labels for XGBoost
le_filtered_y = LabelEncoder()
y_filtered_encoded = le_filtered_y.fit_transform(df_filtered_common['listed_in'])

# Define num_total_classes based on the re-encoded, filtered labels
num_total_classes = len(le_filtered_y.classes_)

# For numerical model
X_num_filtered = df_filtered_common.drop(columns=['listed_in', 'title']) # Drop 'title' here for numerical features

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_num_filtered)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_filtered_encoded, test_size=0.2, random_state=42, stratify=y_

# Ensure y_train and y_test are numpy arrays with contiguous labels for XGBoost
y_train = np.array(y_train)
y_test = np.array(y_test)

# To make text features available for NLP, we need the 'title' column from df_filtered_common
# Let's save the filtered title and target for NLP separately
X_nlp_text = df_filtered_common['title']
y_nlp_target = y_filtered_encoded # Use the same filtered and encoded target for NLP

# Diagnostic prints
```

```
print(f"Minimum class count in y_filtered after encoding: {pd.Series(y_nlp_target).value_counts().min()}")
print(f"Number of samples in y_filtered after encoding: {len(y_nlp_target)}")
print(f"Number of unique classes in y_filtered after encoding: {len(le_filtered_y.classes_)}")
print(f"y_train unique values: {np.unique(y_train)}")
print(f"y_test unique values: {np.unique(y_test)}")
print(f"Max class in y_train: {np.max(y_train)}, Max class in y_test: {np.max(y_test)}")
print(f"num_total_classes: {num_total_classes}")
```

```
Minimum class count in y_filtered after encoding: 2
Number of samples in y_filtered after encoding: 5350
Number of unique classes in y_filtered after encoding: 313
y_train unique values: [  0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
  18  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35
  36  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53
  54  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71
  72  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89
  90  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107
 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143
 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161
 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179
 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197
 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215
 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233
 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251
 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269
 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287
 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305
 306 307 308 309 310 311 312]
y_test unique values: [  0   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
  19  20  21  22  24  25  27  28  29  30  31  33  34  35  36  37  38  40
  41  43  44  45  46  47  48  49  50  51  52  53  54  56  57  58  59  60
  61  63  65  66  67  68  70  71  74  76  77  78  80  81  82  83  84  85
  86  87  90  92  96  97  99 101 105 107 110 111 112 113 114 115 116 117
 118 119 120 121 122 123 124 126 127 129 130 131 132 133 135 138 139 140
 141 145 146 147 148 149 151 153 154 156 157 158 160 161 162 164 165 166
 167 169 170 171 172 173 174 175 176 178 179 180 182 184 185 186 187 188
 190 192 193 194 195 196 197 199 201 202 203 205 206 207 208 209 214 216
 217 219 220 221 222 223 224 226 227 229 230 232 234 236 240 241 242 244
 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262
 263 264 266 267 268 269 270 272 273 274 275 276 277 278 279 280 281 282
 283 284 285 286 288 290 291 292 293 296 297 298 299 300 301 302 303 304
 305 306 307 308 311 312]
Max class in y_train: 312, Max class in y_test: 312
num_total_classes: 313
```

```
log_model = LogisticRegression()
log_model.fit(X_train, y_train)
pred_log = log_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred_log))
print(classification_report(y_test, pred_log))
```

```
           284       0.00      0.00      0.00         2
           285       0.00      0.00      0.00         1
           286       0.00      0.00      0.00         1
           288       0.00      0.00      0.00         3
           290       0.00      0.00      0.00         1
           291       0.00      0.00      0.00         2
           292       0.00      0.00      0.00        11
           293       0.00      0.00      0.00         1
           295       0.00      0.00      0.00         0
           296       0.00      0.00      0.00         1
           297       0.00      0.00      0.00         1
           298       0.00      0.00      0.00         1
           299       0.00      0.00      0.00         1
           300       0.00      0.00      0.00         1
           301       0.00      0.00      0.00         1
           302       0.00      0.00      0.00         4
           303       0.00      0.00      0.00         2
           304       0.00      0.00      0.00         1
           305       0.00      0.00      0.00         1
           306       0.00      0.00      0.00         2
           307       0.17      0.75      0.27        53
           308       0.00      0.00      0.00         1
           311       0.00      0.00      0.00         1
           312       0.00      0.00      0.00        10

       accuracy                           0.12      1070
      macro avg       0.02      0.03      0.02      1070
   weighted avg       0.05      0.12      0.06      1070
```

```
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
pred_dt = dt_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred_dt))
```

```
Accuracy: 0.1383177570093458
```

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf_model.fit(X_train, y_train)

pred_rf = rf_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred_rf))
print(classification_report(y_test, pred_rf))
```

```
           288       0.00      0.00      0.00         3
           290       0.00      0.00      0.00         1
           291       0.00      0.00      0.00         2
           292       0.00      0.00      0.00        11
           293       0.00      0.00      0.00         1
           296       0.00      0.00      0.00         1
           297       0.00      0.00      0.00         1
           298       0.00      0.00      0.00         1
           299       0.00      0.00      0.00         1
           300       0.00      0.00      0.00         1
           301       0.00      0.00      0.00         1
           302       0.50      0.25      0.33         4
           303       0.00      0.00      0.00         2
           304       0.00      0.00      0.00         1
           305       0.00      0.00      0.00         1
           306       0.00      0.00      0.00         2
           307       0.61      0.83      0.70        53
           308       0.50      1.00      0.67         1
           311       0.00      0.00      0.00         1
           312       0.14      0.20      0.17        10

    accuracy                           0.20      1070
   macro avg       0.08      0.09      0.08      1070
weighted avg       0.17      0.20      0.17      1070
```

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Initialize XGBoost model
xgb_model = XGBClassifier(
    objective='multi:softmax',
    eval_metric='mlogloss',
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    random_state=42,
    num_class=num_total_classes # Explicitly set the total number of classes
)

# Train model
xgb_model.fit(X_train, y_train)

# Predict
y_pred = xgb_model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("XGBoost Accuracy:", accuracy)

# Classification Report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# Confusion Matrix
print("\nConfusion Matrix:\n")
print(confusion_matrix(y_test, y_pred))
```

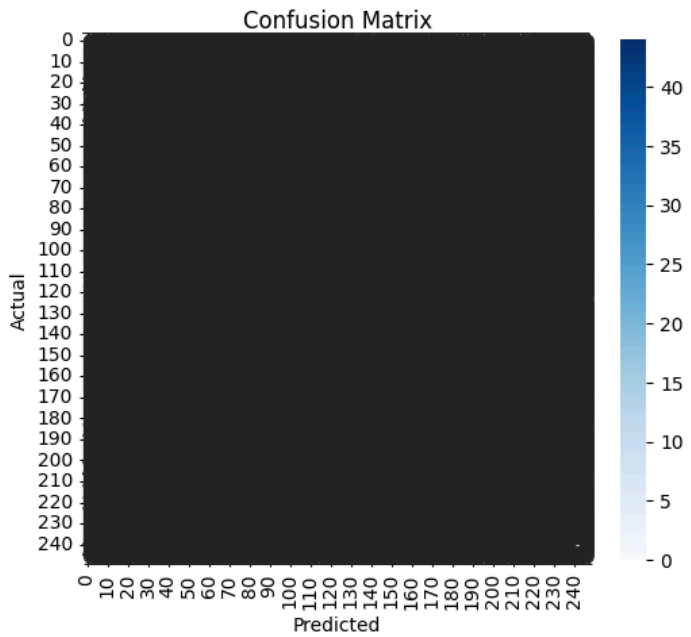
```
           287       0.00      0.00      0.00         0
           288       0.00      0.00      0.00         3
           290       0.00      0.00      0.00         1
           291       0.00      0.00      0.00         2
           292       0.17      0.09      0.12        11
           293       0.00      0.00      0.00         1
           296       0.00      0.00      0.00         1
           297       0.00      0.00      0.00         1
           298       0.00      0.00      0.00         1
           299       0.00      0.00      0.00         1
           300       0.00      0.00      0.00         1
           301       0.00      0.00      0.00         1
           302       0.20      0.25      0.22         4
           303       0.00      0.00      0.00         2
           304       0.00      0.00      0.00         1
           305       0.00      0.00      0.00         1
           306       0.00      0.00      0.00         2
           307       0.68      0.83      0.75        53
           308       0.33      1.00      0.50         1
           311       0.00      0.00      0.00         1
           312       0.17      0.20      0.18        10

    accuracy                           0.21      1070
   macro avg       0.07      0.08      0.07      1070
weighted avg       0.17      0.21      0.18      1070


Confusion Matrix:

[[3 0 0 ... 0 0 0]
 [0 4 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 1 0 0]
 [0 1 0 ... 0 0 0]
 [1 0 0 ... 0 0 2]]
```

```
scores = cross_val_score(rf_model, X_scaled, y_filtered_encoded, cv=5)
print("Cross Validation Score:", scores.mean())
```

```
Cross Validation Score: 0.20934579439252338
```

```
param_grid = {
    'n_estimators':[100,200,300],
    'max_depth':[5,10,20]
}
grid = GridSearchCV(
    RandomForestClassifier(),
    param_grid,
    cv=3
)
grid.fit(X_train, y_train)

best_model = grid.best_estimator_
```

```
cm = confusion_matrix(y_test, pred_rf)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

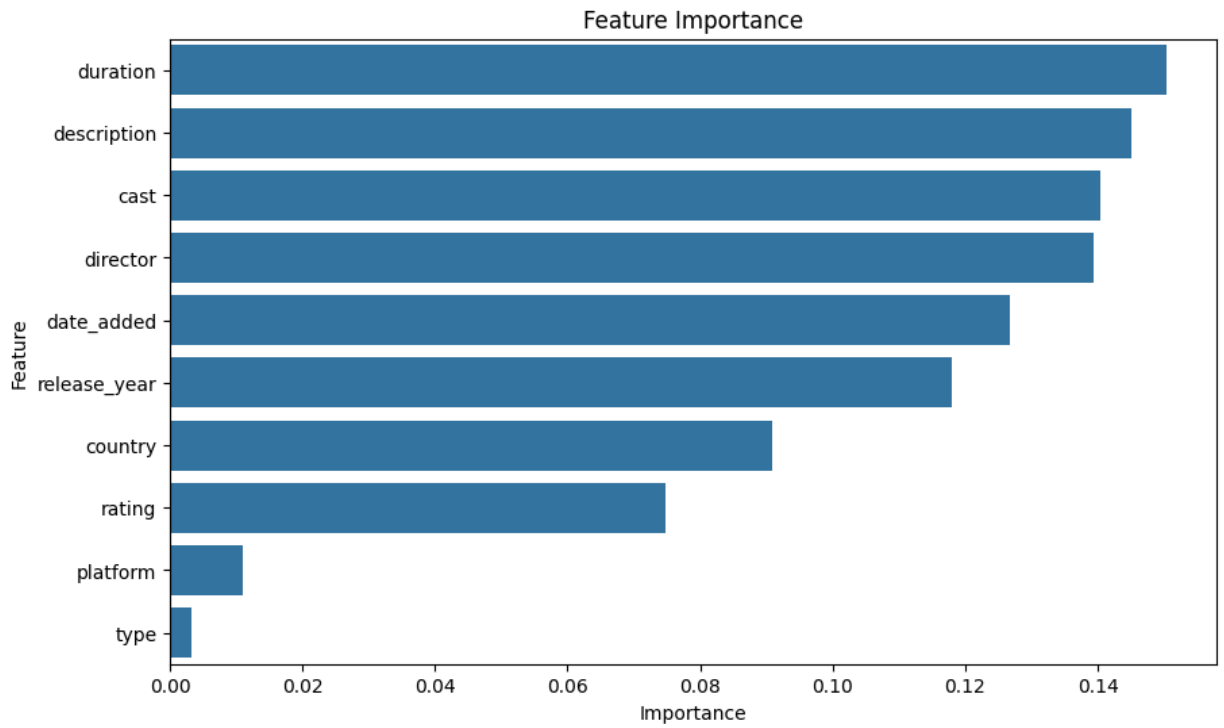



```
importance = rf_model.feature_importances_
features = X_num_filtered.columns
importance_df = pd.DataFrame({
    'Feature':features,
    'Importance':importance
})
importance_df = importance_df.sort_values(by='Importance',ascending=False)
importance_df
```

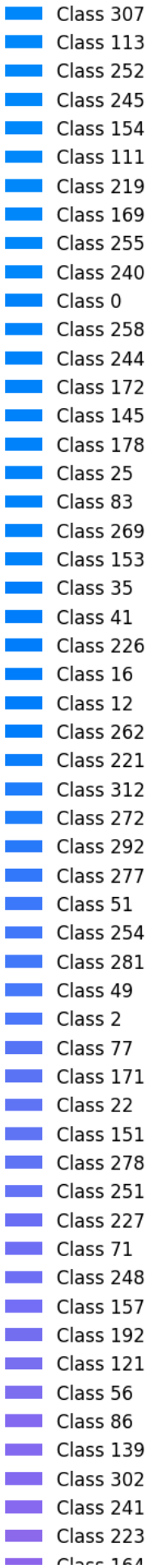
	Feature	Importance
7	duration	0.150417
8	description	0.145070
2	cast	0.140408
1	director	0.139377
4	date_added	0.126659
5	release_year	0.117910
3	country	0.090955
6	rating	0.074886
9	platform	0.010990
0	type	0.003329

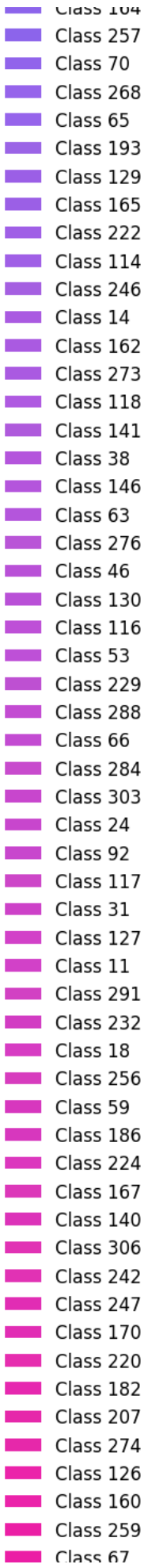
Next steps: Generate code with `importance_df` New interactive sheet

```
plt.figure(figsize=(10,6))
sns.barplot(x='Importance', y='Feature', data=importance_df)
plt.title("Feature Importance")
plt.show()
```



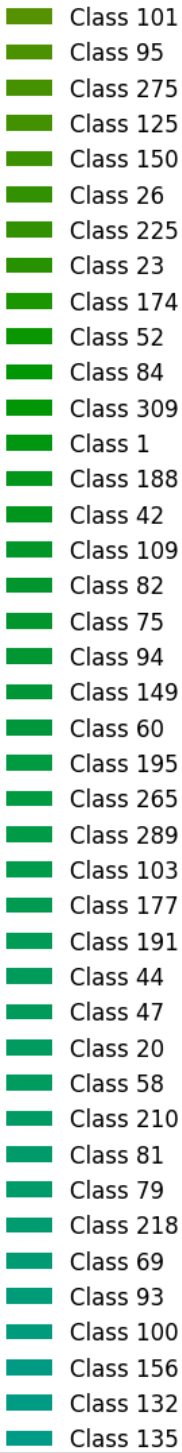

```
import shap

X_sample = shap.sample(X_test, 100)

explainer = shap.TreeExplainer(rf_model)

shap_values = explainer.shap_values(X_sample)

shap.summary_plot(shap_values, X_sample, plot_type="bar")
```


Class 307
Class 113
Class 252
Class 245
Class 154
Class 111
Class 219
Class 169
Class 255
Class 240
Class 0
Class 258
Class 244
Class 172
Class 145
Class 178
Class 25
Class 83
Class 269
Class 153
Class 35
Class 41
Class 226
Class 16
Class 12
Class 262
Class 221
Class 312
Class 272
Class 292
Class 277
Class 51
Class 254
Class 281
Class 49
Class 2
Class 77
Class 171
Class 22
Class 151
Class 278
Class 251
Class 227
Class 71
Class 248
Class 157
Class 192
Class 121
Class 56
Class 86
Class 139
Class 302
Class 241
Class 223


Class 104
Class 257
Class 70
Class 268
Class 65
Class 193
Class 129
Class 165
Class 222
Class 114
Class 246
Class 14
Class 162
Class 273
Class 118
Class 141
Class 38
Class 146
Class 63
Class 276
Class 46
Class 130
Class 116
Class 53
Class 229
Class 288
Class 66
Class 284
Class 303
Class 24
Class 92
Class 117
Class 31
Class 127
Class 11
Class 291
Class 232
Class 18
Class 256
Class 59
Class 186
Class 224
Class 167
Class 140
Class 306
Class 242
Class 247
Class 170
Class 220
Class 182
Class 207
Class 274
Class 126
Class 160
Class 259
Class 67

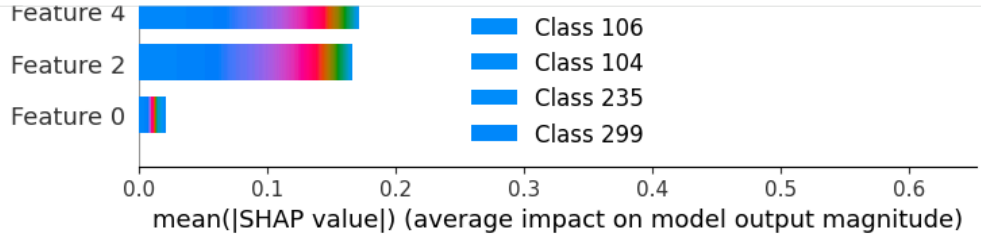
Class 253
Class 85
Class 61
Class 19
Class 250
Class 124
Class 180
Class 122
Class 120
Class 184
Class 209
Class 185
Class 249
Class 176
Class 208
Class 110
Class 199
Class 54
Class 76
Class 87
Class 266
Class 36
Class 304
Class 33
Class 217
Class 196
Class 7
Class 8
Class 216
Class 37
Class 5
Class 280
Class 166
Class 205
Class 138
Class 48
Class 74
Class 28
Class 50
Class 230
Class 17
Class 203
Class 78
Class 147
Class 10
Class 27
Class 21
Class 97
Class 260
Class 4
Class 148
Class 173
Class 187
Class 175
Class 206

Class 34
Class 13
Class 282
Class 285
Class 267
Class 161
Class 270
Class 243
Class 308
Class 264
Class 286
Class 136
Class 197
Class 133
Class 119
Class 134
Class 15
Class 9
Class 102
Class 143
Class 201
Class 261
Class 142
Class 99
Class 32
Class 159
Class 39
Class 57
Class 179
Class 263
Class 236
Class 112
Class 45
Class 228
Class 43
Class 96
Class 183
Class 55
Class 3
Class 158
Class 128
Class 68
Class 283
Class 137
Class 155
Class 237
Class 163
Class 123
Class 271
Class 144
Class 30
Class 234
Class 279
Class 131
Class 29


Class 101
Class 95
Class 275
Class 125
Class 150
Class 26
Class 225
Class 23
Class 174
Class 52
Class 84
Class 309
Class 1
Class 188
Class 42
Class 109
Class 82
Class 75
Class 94
Class 149
Class 60
Class 195
Class 265
Class 289
Class 103
Class 177
Class 191
Class 44
Class 47
Class 20
Class 58
Class 210
Class 81
Class 79
Class 218
Class 69
Class 93
Class 100
Class 156
Class 132
Class 135
Class 90
Class 40
Class 152
Class 296
Class 80
Class 168
Class 204
Class 73
Class 211
Class 190
Class 6
Class 202
Class 198
Class 181

```
pickle.dump(rf_model, open("genre_prediction_model.pkl", "wb"))
```

```
import optuna
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier

def objective(trial):
    param = {
        'objective': 'multi:softmax',
        'eval_metric': 'mlogloss',
        'num_class': num_total_classes,
        'n_estimators': trial.suggest_int('n_estimators', 100, 1000),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'gamma': trial.suggest_float('gamma', 0.0, 0.5),
        'random_state': 42
    }

    model = XGBClassifier(**param)
    model.fit(X_train, y_train, verbose=False)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    return accuracy

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=5) # You can adjust n_trials for more comprehensive search

print("Best trial:")
print(f"  Value: {study.best_value}")
print(f"  Params: ")
for key, value in study.best_params.items():
    print(f"    {key}: {value}")

# Update the existing xgb_model with best parameters
xgb_model_optimized = XGBClassifier(**study.best_params, objective='multi:softmax', eval_metric='mlogloss', num_class=num_tot
xgb_model_optimized.fit(X_train, y_train)
y_pred_optimized = xgb_model_optimized.predict(X_test)

print("\nOptimized XGBoost Accuracy:", accuracy_score(y_test, y_pred_optimized))
print("\nOptimized Classification Report:\n")
print(classification_report(y_test, y_pred_optimized))
print("\nOptimized Confusion Matrix:\n")
print(confusion_matrix(y_test, y_pred_optimized))
```



```
           301       0.00      0.00      0.00         1
           302       0.20      0.25      0.22         4
           303       0.00      0.00      0.00         2
           304       0.00      0.00      0.00         1
           305       0.00      0.00      0.00         1
           306       0.00      0.00      0.00         2
           307       0.67      0.89      0.76        53
           308       1.00      1.00      1.00         1
           311       0.00      0.00      0.00         1
           312       0.07      0.10      0.08        10

    accuracy                           0.22      1070
   macro avg       0.08      0.09      0.08      1070
weighted avg       0.18      0.22      0.19      1070


Optimized Confusion Matrix:

[[6 0 0 ... 0 0 2]
 [0 4 0 ... 0 0 0]
 [0 0 0 ... 0 0 0]
 ...
 [0 0 0 ... 1 0 0]
 [0 0 0 ... 0 0 0]
 [1 0 0 ... 0 0 1]]
```

```
requirements = """
pandas
numpy
scikit-learn
matplotlib
seaborn
xgboost
shap
streamlit
joblib
nltk
"""

with open("requirements.txt","w") as f:
    f.write(requirements)
```

```
!pip freeze > requirements.txt
```

```
from google.colab import files
files.download("requirements.txt")
```

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer

# Combine text columns
df_filtered_common['text'] = (
    df_filtered_common['title'].astype(str) + " " +
    df_filtered_common['description'].astype(str)
)

X_nlp_text = df_filtered_common['text'].fillna("")

# Clean text
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'[^a-zA-Z0-9 ]', '', text)
    return text

X_nlp_text = X_nlp_text.apply(clean_text)

# Remove empty rows
mask = X_nlp_text.str.strip() != ""
X_nlp_text = X_nlp_text[mask]
y_nlp_target = y_filtered_encoded[mask]

# TF-IDF
vectorizer = TfidfVectorizer(
```

```
    max_features=5000,
    stop_words='english',
    min_df=2
)

X_text_vectorized = vectorizer.fit_transform(X_nlp_text)
```

```
!pip install streamlit
```

```
Collecting streamlit
  Downloading streamlit-1.55.0-py3-none-any.whl.metadata (9.8 kB)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.5
Requirement already satisfied: blinker<2,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (1.9.0)
Requirement already satisfied: cachetools<8,>=5.5 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.2.6)
Requirement already satisfied: click<9,>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (8.3.1)
Requirement already satisfied: gitpython!=3.1.19,<4,>=3.0.7 in /usr/local/lib/python3.12/dist-packages (from streamlit) (3.1.4
Requirement already satisfied: numpy<3,>=1.23 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.0.2)
Requirement already satisfied: packaging>=20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (26.0)
Requirement already satisfied: pandas<3,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.2.2)
Requirement already satisfied: pillow<13,>=7.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (11.3.0)
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
  Downloading pydeck-0.9.1-py2.py3-none-any.whl.metadata (4.1 kB)
Requirement already satisfied: protobuf<7,>=3.20 in /usr/local/lib/python3.12/dist-packages (from streamlit) (5.29.6)
Requirement already satisfied: pyarrow>=7.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (18.1.0)
Requirement already satisfied: requests<3,>=2.27 in /usr/local/lib/python3.12/dist-packages (from streamlit) (2.32.4)
Requirement already satisfied: tenacity<10,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (9.1.4)
Requirement already satisfied: toml<2,>=0.10.1 in /usr/local/lib/python3.12/dist-packages (from streamlit) (0.10.2)
Requirement already satisfied: tornado!=6.5.0,<7,>=6.0.3 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.5.1)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from streamlit) (4.15.
Requirement already satisfied: watchdog<7,>=2.1.5 in /usr/local/lib/python3.12/dist-packages (from streamlit) (6.0.0)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0->streaml
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.0
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<7,>=4.
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from gitpython!=3.1.19,<4,>=3.0.7->
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->strea
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (202
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (2
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->st
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit) (3.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamli
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamli
Requirement already satisfied: smmap<6,>=3.0.1 in /usr/local/lib/python3.12/dist-packages (from gitdb<5,>=4.0.1->gitpython!=3.
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->altair!=5.4.0,!=5.4.1,
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4.0,!
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschem
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=3.0->altair!=5.4.0
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas<3,>=1.
Downloading streamlit-1.55.0-py3-none-any.whl (9.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.1/9.1 MB 56.4 MB/s eta 0:00:00
Downloading pydeck-0.9.1-py2.py3-none-any.whl (6.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 78.7 MB/s eta 0:00:00
Installing collected packages: pydeck, streamlit
Successfully installed pydeck-0.9.1 streamlit-1.55.0
```

```
import streamlit as st
import pickle
import numpy as np

# Placeholder for the actual model path and label encoder
# For deployment, ensure 'genre_prediction_model.pkl' and 'label_encoder.pkl' are available
# and that the input features match the model's expectations.

# Mock model and encoder for demonstration if files aren't present
try:
    model = pickle.load(open("genre_prediction_model.pkl","rb"))
    # Assuming label_encoders contains the encoder for 'listed_in'
    # le_filtered_y would be needed to convert numerical predictions back to genre names
    # For this example, we'll just return the number.
except FileNotFoundError:
    st.warning("Model or label encoder not found. Using a dummy model for demonstration.")
    class DummyModel:
        def predict(self, features):
            # Dummy prediction logic (e.g., always predict 0)
            return np.array([0])
    model = DummyModel()

st.title("Movie Genre Prediction App")
```